Push Button Controlled Buzzer using Arduino Uno

Platform: Wokwi simulation | Board: Arduino Uno | Author: Akash

1. Aim

To make a buzzer sound only while a push button is held down, and stop the moment the button is released.

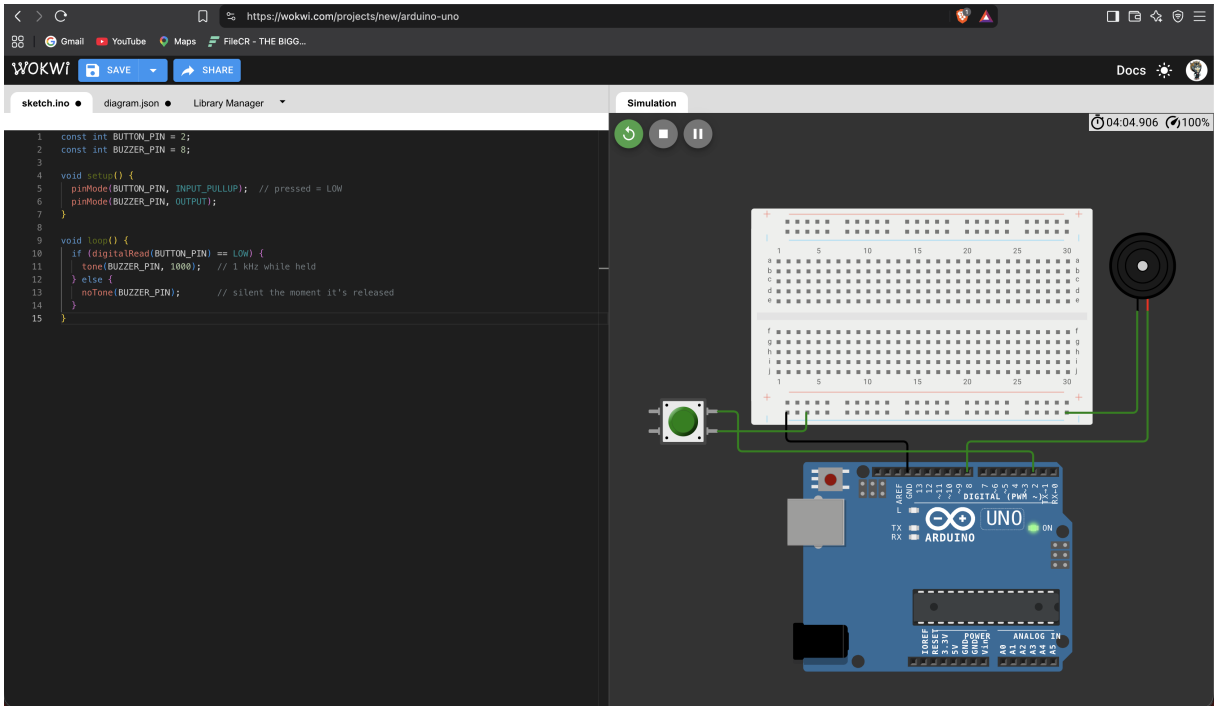
2. Components

Component	Qty
Arduino Uno	1
Push button	1
Buzzer	1
Breadboard	1
Jumper wires	As required
Resistor / capacitor	Not needed (see section 4)

3. Connections

From	To
Push button, one leg	Arduino pin D2
Push button, other leg	Arduino GND
Buzzer positive (+)	Arduino pin D8
Buzzer negative (-)	Arduino GND

Simulation screenshot (Wokwi):



Wokwi project link: (paste your share link here)

4. Why I used pins D2 and D8

Why D2 for the button. Pins D0 and D1 are used by the USB serial connection (TX and RX), so using them for your own parts can interfere with uploading code. D2 is the first free digital pin after those two, so it is the natural starting point. D2 is also one of only two pins (D2 and D3) that support hardware interrupts, so if the project is extended later (for example, reacting to the button without checking it constantly), the wiring would not need to change.

Why D8 for the buzzer. D8 is a plain digital pin. It has no special job: it is not used by serial (D0, D1), not used by SPI (D10 to D13), and not tied to the onboard LED (D13). Keeping the buzzer on a pin with no other duty avoids conflicts. The `tone()` function works on any digital pin, so D8 is fine.

Honest note: neither pin is compulsory. Any free digital pin would work for this circuit. These two were chosen because they are safe, simple, and keep the wiring tidy.

Why no resistor is needed. The button uses the Arduino's built-in pull-up resistor (turned on in code with `INPUT_PULLUP`), so no external resistor is required. The simulated buzzer draws very little current, so it can be driven directly from D8.

5. Complete code

```
const int BUTTON_PIN = 2;
const int BUZZER_PIN = 8;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // pressed = LOW
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    tone(BUZZER_PIN, 1000);   // 1 kHz while held
  } else {
    noTone(BUZZER_PIN);       // silent the moment it's released
  }
}
```

6. Code explained line by line (for beginners)

An Arduino program is called a **sketch**. Every sketch has two main parts: `setup()`, which runs once at the start, and `loop()`, which then repeats forever.

Code	What it means
`const int BUTTON_PIN = 2;`	Creates a name, *BUTTON_PIN*, and gives it the value 2. *int* means a whole number. *const* means the value will never change while the program runs. We use a name instead of writing 2 everywhere so the code is easier to read, and if the wire moves to another pin only this one line needs changing.
`const int BUZZER_PIN = 8;`	Same idea. The buzzer is connected to pin 8, so *BUZZER_PIN* stands for 8.
`void setup() {`	Starts the setup function. *void* means it gives back no value. The curly bracket { opens the block of instructions. Everything inside runs **one time** when the Arduino powers on or is reset.
`pinMode(BUTTON_PIN, INPUT_PULLUP);`	Tells the Arduino how to use pin 2. *INPUT* means it will read a signal. *PULLUP* switches on a small resistor inside the chip that holds the pin at HIGH (5 V) when nothing is pressed. When the button is pressed it connects the pin to GND, so the pin reads LOW. This is why **pressed = LOW**. The comment after // is only a note for humans; the Arduino ignores it.
`pinMode(BUZZER_PIN, OUTPUT);`	Tells the Arduino that pin 8 will *send* a signal out, so it can drive the buzzer.
`}`	Closes the setup function.
`void loop() {`	Starts the loop function. After setup finishes, the Arduino runs everything inside this block, then jumps back to the top and runs it again, forever. It repeats thousands of times per second.

Code	What it means
`if (digitalRead(BUTTON_PIN) == LOW) {`	*digitalRead(BUTTON_PIN)* checks pin 2 and returns either HIGH or LOW. == means "is equal to" (a single = would assign a value, so always use two to compare). So this line asks: "is the button pressed right now?" If yes, run the lines inside the brackets that follow.
`tone(BUZZER_PIN, 1000);`	Makes pin 8 switch on and off 1000 times per second (1000 Hz, or 1 kHz). That vibration is what you hear as a beep. The Arduino does this in the background using a timer, so the code does not have to do it manually.
`} else {`	Closes the "if" part and starts the alternative. Everything inside the else block runs when the "if" condition was *not* true, meaning the button is **not** pressed.
`noTone(BUZZER_PIN);`	Stops the tone on pin 8, so the buzzer goes silent.
`}`	Closes the else block.
`}`	Closes the loop function. The Arduino now goes back to the start of loop() and checks the button again.

7. How the whole thing works

Situation	Pin D2 reads	Result on D8
Button not pressed	HIGH	Silent (noTone)
Button held down	LOW	1 kHz tone (buzzer sounds)
Button released	HIGH again	Silent again

Because loop() repeats so quickly and the code has no delays, the buzzer reacts instantly: it starts as soon as the button is pressed and stops as soon as it is released.

8. Conclusion

The circuit uses only a push button and a buzzer. The internal pull-up resistor removes the need for external components, and a simple if/else in loop() makes the buzzer follow the button exactly.